

Egyptian Currency Detection — Project Report

Date: 2026-05-02 **Author:** MazenBasha (graduation project) **Repo path:**

D:\roshdi-grad-project\Egyptian Currency Detection

1. Goal

Upgrade an existing single-image Egyptian-currency *classifier* into a *multi-note detection* pipeline that can:

- Detect every visible note in an image / video / live camera frame.
- Classify each note (1, 5, 10, 20, 50, 100, 200 EGP).
- Count the notes per denomination.
- Compute the total amount of money in view.
- Emit annotated media (boxes + labels + summary panel) and a JSON record.

Target JSON output:

```
{
  "detections": [
    {"class": "100_EGP", "confidence": 0.94, "bbox": [x1, y1, x2, y2]},
    {"class": "100_EGP", "confidence": 0.91, "bbox": [x1, y1, x2, y2]},
    {"class": "50_EGP", "confidence": 0.88, "bbox": [x1, y1, x2, y2]}
  ],
  "counts": { "100_EGP": 2, "50_EGP": 1 },
  "total": 250
}
```

2. Starting point — what existed

The original repo contained a from-scratch image-classification model only:

File	Role
model.py	CurrencyMobileNet (~2.2M params, MobileNetV2-style)
dataset.py	Folder-per-class loader (data/<split>/<class>/*.jpg)
train.py	Training loop with AMP, weighted sampling, label smoothing
evaluate.py	Accuracy / precision / recall / F1 / confusion matrix
infer.py	Single-image, single-class inference
camera.py	OpenCV contour detection + classification per region — fragile
export_ptl.py	TorchScript Lite export for mobile
outputs/best_model.pth	Trained classifier checkpoint

Classes (9): 1, 5, 10, 10 (new), 20, 20 (new), 50, 100, 200.

Why this couldn't be used to count multiple notes: the head is a single softmax over the full image. There is no spatial output, and the training data has no bounding boxes — the model only knows *what kind* of single note fills the frame, never *where* notes are.

3. Architectural decision

Two paths were considered:

Path	Pros	Cons
A — Hybrid (region proposal + existing classifier)	No new labeling	Fragile on cluttered backgrounds, overlap, partial visibility; no proper detection metrics
B — YOLOv8 detector (chosen)	Native multi-note detection; handles overlap; real mAP/P/R metrics; one model end-to-end	Needs bounding-box labels (not present in the original dataset)

Decision: Option B — YOLOv8.

To bootstrap without manual labeling, we generated **pseudo-labels** (one full-image box per training image, class = source folder). This lets the model train immediately on single-note photos. The trade-off — and its consequence on real multi-note inference — is documented in §8.

4. Project reorganization

```

Egyptian Currency Detection/
■■■■ src/
■   ■■■■ config.py
■   ■■■■ train.py
■   ■■■■ detect.py
■   ■■■■ evaluate.py
■   ■■■■ utils.py
■   ■■■■ bootstrap_yolo_dataset.py
■■■■ data_yolo/
■   ■■■■ data.yaml
■   ■■■■ images/{train,val,test}/
■   ■■■■ labels/{train,val,test}/
■■■■ outputs/
■   ■■■■ runs/train2/
■   ■■■■ runs/eval/
■   ■■■■ detections/
■■■■ legacy/
■   ■■■■ data/
■   ■■■■ model.py, train.py, ...
■■■■ requirements.txt
■■■■ README.md
■■■■ PROJECT_REPORT.md

# NEW — all working code lives here
# paths, classes, hyperparameters
# YOLOv8 training (with --low-mem)
# image / folder / video / webcam
# mAP / P / R / confusion matrix
# box drawing, count + total, JSON
# converts classification folders -> YOLO

# YOLO dataset spec (7 classes)
# *.jpg
# *.txt (YOLO format)

# training run (best.pt, last.pt, plots)
# test-split evaluation results
# annotated images + JSON output
# original classifier project (untouched)
# original folder-per-class dataset
# all original files

# full setup + usage guide
# this document

```

The original project was preserved as-is in `legacy/` — nothing was deleted.

5. Class scheme

The original dataset has 9 folders (old & new variants of 10 EGP and 20 EGP are separate). For monetary counting these collapse to **7 classes** since old/new have the same value:

Source folder(s)	YOLO ID	Class name	Value (EGP)
1	0	1_EGP	1
5	1	5_EGP	5
10, 10 (new)	2	10_EGP	10
20, 20 (new)	3	20_EGP	20
50	4	50_EGP	50
100	5	100_EGP	100
200	6	200_EGP	200

6. Dataset

Source dataset (folder-per-class, no boxes): e:\data\egyptian_currency\data.

Bootstrap step (src/bootstrap_yolo_dataset.py):

- Copies every image into data_yolo/images/{train, val, test}/ (flat, filenames prefixed with the source folder slug to avoid collisions).
- Generates a YOLO .txt for each image with a single full-image box at the mapped class id (<id> 0.5 0.5 1.0 1.0).
- Merges old/new variants into the 7-class scheme above.

Resulting splits:

Split	Images	Notes
Train	2637	one note per image
Val	760	one note per image
Test	290	one note per image
Total	3687	

Training images per class (post-merge):

Class	Approx. count
1_EGP	60
5_EGP	334
10_EGP	~632 (315 + 317)
20_EGP	~668 (322 + 346)
50_EGP	315
100_EGP	315
200_EGP	313

7 corrupt JPEGs were auto-restored by ultralytics during the first scan.

The original folder-style dataset is also preserved at `legacy/data/` for the original classifier.

7. Training

Tooling: `ultralytics==8.4.46`, `torch==2.11.0+cpu`, Python 3.11.9. Hardware: **CPU only** (Intel Core i7-9750H @ 2.60 GHz, 6 cores). No CUDA GPU was available.

Run history

- **Run 1** (`b1s0b3nod`) — failed at startup: `data.yaml` used `path: .`

which ultralytics resolved against its global `datasets_dir` rather than the yaml file. **Fix:** switched `path` to an absolute path.

- **Run 2** (`b9os1pwo7`) — started cleanly with `batch=8`, `imgsz=480`, mosaic +

mixup on. Reached **3 completed epochs**, then crashed at epoch 4 with `RuntimeError: not enough memory: tried to allocate 7372800 bytes` during the head's forward pass. CPU RAM ran out under mosaic augmentation.

- **Run 3** (`b0vskux83`) — resumed from `last.pt` of run 2 with reduced

memory footprint:

- batch **8** → **4**
- `imgsz` **480** → **416**
- **mosaic + mixup + copy-paste disabled** (added `--low-mem` flag to `src/train.py`)
- 22 additional epochs
- Saved under `outputs/runs/train2/`. **Completed successfully.**

Hyperparameters (run 3)

Parameter	Value
Base model	yoLov8n.pt (3.0M params, 8.2 GFLOPs)
Optimizer	AdamW (auto-selected; lr0=0.000909, momentum=0.9)
LR schedule	Cosine, lrf=0.01
Epochs	22 (continuing from run 2)
Batch	4
Image size	416
HSV jitter	h=0.015, s=0.7, v=0.5
Geometric	rotation $\pm 20^\circ$, translate 0.1, scale 0.5, shear 2° , perspective 0.0005
Flips	flipLR 0.5, flipUD 0.1
Mosaic / Mixup / Copy-Paste	disabled (low-mem)
Patience	20
Seed	42

Training time

- Run 2: ~25 minutes (3 epochs)
- Run 3: **2 hours 32 minutes** (22 epochs)
- **Total wall-clock:** ~3 hours

Validation curve (per epoch, from results.csv)

Epoch	Box loss (train)	Cls loss (train)	mAP50	mAP50-95	P	R
1	0.170	1.035	0.939	0.913	0.831	0.893
5	0.155	0.673	0.952	0.886	0.893	0.853
10	0.119	0.397	0.986	0.961	0.957	0.967
12	0.107	0.312	0.993	0.972	0.966	0.973
15	0.089	0.218	0.993	0.979	0.977	0.985
17	0.081	0.171	0.995	0.985	0.986	0.979
20	0.066	0.140	0.994	0.989	0.989	0.971
22	0.062	0.129	0.995	0.989	0.991	0.964

Loss curves and PR / F1 / confusion-matrix plots are saved under `outputs/runs/train2/` (`results.png`, `BoxPR_curve.png`, `BoxF1_curve.png`, `confusion_matrix.png`, `confusion_matrix_normalized.png`, `val_batch*_labels.jpg`, `val_batch*_pred.jpg`).

Final weights:

- `outputs/runs/train2/weights/best.pt` — 6.2 MB

- `outputs/runs/train2/weights/last.pt` — 6.2 MB

8. Test-set evaluation

Run: `python src/evaluate.py --split test --imgsz 416 --batch 4 --device cpu`

Results saved under `outputs/runs/eval/`.

Metric	Value
Test images	290
Total instances	290
mAP@0.5	0.9949
mAP@0.5:0.95	0.9894
Precision	0.9911
Recall	0.9852
Inference latency (CPU)	~27 ms / image

Per-class on test split

Class	Instances	mAP50	mAP50-95	Precision	Recall
1_EGP	20	0.9950	0.9950	0.9595	1.0000
5_EGP	35	0.9950	0.9923	1.0000	0.9539
10_EGP	65	0.9950	0.9880	1.0000	0.9945
20_EGP	65	0.9950	0.9789	0.9996	1.0000
50_EGP	35	0.9950	0.9950	0.9923	1.0000
100_EGP	35	0.9944	0.9816	1.0000	0.9479
200_EGP	35	0.9950	0.9950	0.9866	1.0000

Every class achieves $\text{mAP}@0.5 \geq 0.994$. The lowest recall is on 100_EGP (0.948), which the confusion matrix shows is occasionally confused with 50_EGP — likely due to color similarity in low-quality training photos.

9. End-to-end pipeline test

`detect.py` was run on a sample of test images.

Input image	Detections	Counts	Total
1__1.100.jpg	1 box	1_EGP: 1	1 EGP ✓
100__100.100.jpg	2 boxes	100_EGP: 2	200 EGP ■ (image actually has 1 note)
200__200.10.jpg	2 boxes	200_EGP: 2	400 EGP ■ (image actually has 1 note)
50__50.1.jpg	2 boxes	50_EGP: 2	100 EGP ■ (image actually has 1 note)

Each run produced an annotated JPG and a JSON file matching the target schema (outputs/detections/<stem>_annotated.jpg, <stem>.json).

The duplication artefact

On 3 of 4 spot-checks the model produced **two overlapping boxes around a single physical note**. mAP doesn't penalize this (NMS at 0.5 IoU + the metric's tolerance let both boxes "match" the same ground-truth box), but the **counter and the running total are wrong**.

Root cause: every training label was a *full-image pseudo-box*. The network never saw "one note ≠ entire frame," so at inference it confidently emits boxes that cover most of the frame, plus second slightly-shifted boxes that survive class-aware NMS.

Implication: the 0.995 mAP and the working JSON pipeline are real, but they only validate single-note classification — not the actual multi-note counting use case. To fix the duplication for multi-note photos, real hand-drawn bounding-box labels are required.

10. Deliverables

Code (all in `src/`, all newly written)

File	Purpose
<code>config.py</code>	Single source of truth: paths, 7 classes, denominations, hyperparameters
<code>train.py</code>	Wraps <code>ultralytics</code> ; currency-tuned augmentation; -- low-mem, -- resume flags
<code>detect.py</code>	Image / folder / video / webcam → annotated media + JSON
<code>evaluate.py</code>	mAP50, mAP50-95, P, R, per-class breakdown, confusion matrix
<code>utils.py</code>	Result parsing, box drawing, count + total, JSON writer
<code>bootstrap_yolo_dataset.py</code>	Converts classification folder dataset → YOLO format

Trained model

- `outputs/runs/train2/weights/best.pt` (6.2 MB)
- `outputs/runs/train2/weights/last.pt` (6.2 MB)

Results / plots

- Training curves: `outputs/runs/train2/results.png`
- PR curve: `outputs/runs/train2/BoxPR_curve.png`
- F1 curve: `outputs/runs/train2/BoxF1_curve.png`
- Confusion matrix: `outputs/runs/train2/confusion_matrix.png` and `confusion_matrix_normalized.png`
- Validation predictions: `outputs/runs/train2/val_batch*_pred.jpg`
- Per-epoch CSV: `outputs/runs/train2/results.csv`
- Test evaluation: `outputs/runs/eval/`
- Sample detections + JSON: `outputs/detections/`

Documentation

- `README.md` — end-to-end setup + commands
- `PROJECT_REPORT.md` — this document

11. How to run

```
# 1. Install
pip install -r requirements.txt

# 2. Build the YOLO dataset from the original classification folders
python src/bootstrap_yolo_dataset.py --src "e:\data\egyptian_currency\data"

# 3. Train (CPU-friendly; drop --low-mem on a GPU)
python src/train.py --epochs 25 --batch 4 --imgsz 416 --device cpu --low-mem

# 4. Evaluate on test split
python src/evaluate.py --split test --imgsz 416 --batch 4 --device cpu

# 5. Run detection
python src/detect.py --image path/to/photo.jpg
python src/detect.py --image-dir path/to/folder
python src/detect.py --video path/to/clip.mp4
python src/detect.py --webcam
```

Each detection produces `<stem>_annotated.jpg` and `<stem>.json` in `outputs/detections/`.

12. Limitations & next steps

Limitation	Impact	Fix
Pseudo-labels (full-image boxes)	Model emits duplicate boxes on real photos → wrong counts/totals	Hand-label ~50–150 multi-note images with real boxes; fine-tune from <code>best.pt</code>
All training images are single-note	No real test of overlap / partial occlusion / multiple notes	Mix multi-note photos into train/val
1_EGP under-represented	60 train images vs 300+ for others	Add more 1_EGP photos
CPU-only training	2.5 h for 22 epochs	Train on GPU; re-enable mosaic + mixup
Mobile export not yet done	Can't deploy on Android/iOS yet	<code>model.export(format="tfLite")</code> or <code>format="onnx"</code> once labels are real

Recommended first follow-up: label ~100 hand-drawn bounding boxes on multi-note photos in Roboflow, drop them into `data_yolo/`, run `python src/train.py --model outputs/runs/train2/weights/best.pt --epochs 30 --low-mem`. The duplicate-box issue should disappear and counting will become reliable.

13. What changed vs. the original project

Aspect	Before	After
Task	Classification (one label per image)	Detection (multi-note, with boxes)
Architecture	CurrencyMobileNet (custom, 2.2M params)	YOLOv8n (pretrained, 3.0M params)
Output	Top-1 / Top-K class	<code>{detections, counts, total}</code> JSON + annotated media
Inputs	Single image	Image, folder, video, webcam
Counting	Not possible	Per-class counts + total EGP value
Metric	Classification accuracy	mAP@0.5, mAP@0.5:0.95, precision, recall
Project layout	Flat	<code>src/</code> + <code>data_yolo/</code> + <code>legacy/</code> separation
Original code	—	Preserved verbatim under <code>legacy/</code>